Expense to account classifier

Peakflo take-home · Muaaz Shaikh · 1 October 2026

1. Executive summary

Peakflo's finance team codes every expense line to one of about a hundred accounting accounts by hand. This project builds a model that does it for them from the information already on the bill: the vendor, the line description, the amount and the date. The approach is deliberately simple. If a vendor has billed the exact same line before, the model reuses that account. Otherwise a small statistical model reads the description and the vendor and picks the most likely account.

Tested fairly, on bills the model had never seen, it picks the right account for **88.9 lines out of 100**, above the 85% target. When it is unsure, it is usually close: the right account is among its top three suggestions 95% of the time. In practice that means the model can post about **seven lines in ten automatically with 98% accuracy**, and hand the remaining three to a reviewer with three suggestions to choose from, instead of the reviewer coding all ten from scratch.

What was measured	Result
Accuracy on unseen bills (5-fold cross-validation, grouped by bill)	88.9% (range 88.0 to 89.7%)
Correct account in the top three suggestions	95.3%
Lines that can be auto-coded at 98% accuracy	71%
Accuracy when trained on the past and tested on the next six months	70.4%

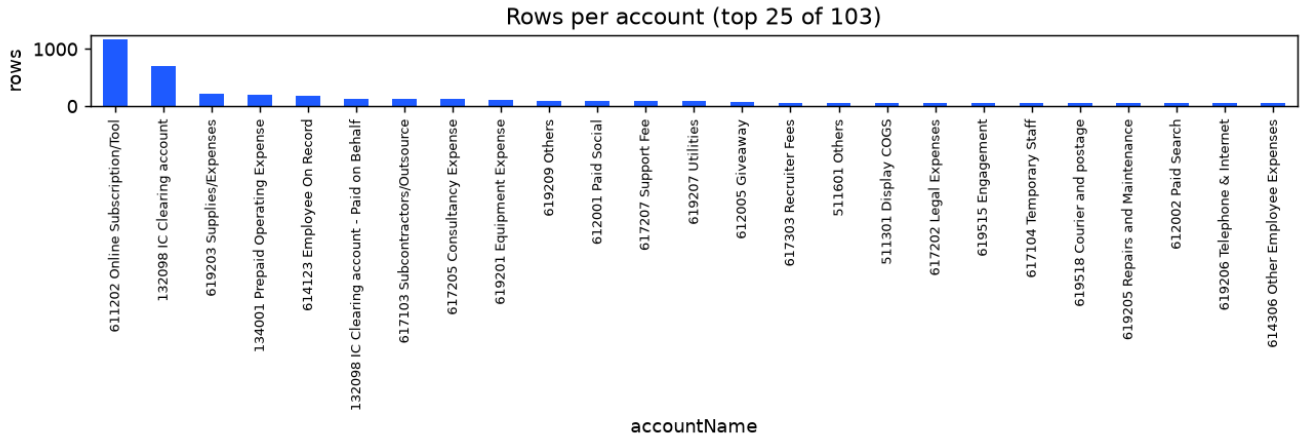
Two findings matter beyond the headline number. First, several accounts were renamed during the period, so the date of the bill is a genuine signal; adding it was worth almost three points. Second, the last row of the table is the honest deployment number: it is lower mainly because one large account was renamed right after the training cutoff and because new vendors keep appearing. Both are process issues, not model issues, and section 5 says what to do about them.

2. Data analysis

Shape of the data. 4,894 rows, 337 vendors, 97 account IDs and 103 account names. The `_id` field is not a line identifier: only 3,181 values are unique, so `_id` is the bill and rows are its lines. 388 bills have lines in more than one account. This matters for validation, because sibling lines share the vendor, the period and often the wording.

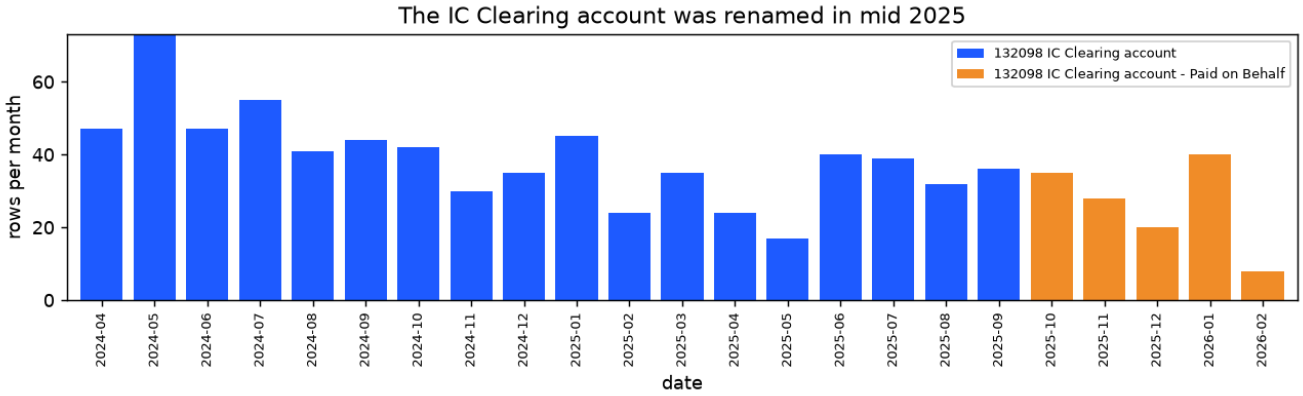
Every row has a date. `_id` is a MongoDB ObjectId, and its first eight hex characters are a creation timestamp. The data runs from April 2024 to February 2026. Item names also usually start with a period code such as `1225` or `0925–1125`, but 532 rows have none, so the ObjectId date is the reliable time key.

Class imbalance is severe. Class imbalance means a few accounts have most of the rows while many accounts have almost none. The largest account, Online Subscription/Tool, is 24% of all rows and the top ten cover about 62%. At the other end, 34 accounts have fewer than 5 rows and 16 appear exactly once. Those 34 accounts hold 68 rows in total, 1.4% of the data, so they barely move overall accuracy but they dominate macro F1 and they cannot be validated in any meaningful way.



Vendor is a strong but incomplete signal. 211 of 337 vendors always map to one account. Predicting each vendor's most common account gives 73.9% on the training data itself and 68.9% out of fold. The remaining points have to come from the text. A few vendors are spread across many accounts: the largest uses 19 accounts over 380 rows and looks like a corporate card or expense claim channel, where only the line text tells you what was bought.

Six accounts were renamed during the period. Six account IDs carry two names each. Plotted over time, these are clean renames, not overlapping categories. "132098 IC Clearing account" runs from April 2024 to September 2025 and "132098 IC Clearing account - Paid on Behalf" starts in October 2025. Lease payable, Employee Training, Customer Prepayment and Trademarks - Cost show the same pattern. Since the target is the name, the date is a genuine signal, and this is the single most useful insight from the exploration.



Text quality. `itemName` equals `itemDescription` in 83% of rows and the description is empty in 31, so the two fields are merged only when they differ. The text is short and keyword-heavy, with vendor product names, entity codes (SG, HK, VN, Mudah, Chotot) and occasional long hardware spec sheets.

Amounts. The median line is SGD 845 and the 90th percentile is about 9,700, but 7 rows exceed one million (a laptop at 108 million is clearly not in SGD) and 19 are negative credit notes. The raw amount is therefore not used directly, only its sign and order of magnitude.

Recurring lines. 58% of rows share their (vendor, cleaned text) pair with at least one other row, and for 51% the other rows all agree on one account. A leave-one-out check of that lookup is 98.5% accurate. Finance data repeats, and a good chunk of the problem is remembering.

3. Methodology

Algorithm choice. With about 5,000 short texts and 103 classes, TF-IDF plus a linear model is the strongest well-understood option. TF-IDF turns each description into a list of the words and word fragments it contains, weighted so that common filler words count less. A linear model then learns how much each word, vendor and

date points towards each account. It trains in seconds, gives probabilities for a confidence gate, and its weights can be read as "top words for this account". A linear SVM scored 0.6 points higher in cross-validation but does not produce probabilities, so logistic regression was kept. Gradient-boosted trees, embeddings and stacking were considered and left out: the brief asks for a simple, well-executed approach, and the time was better spent on evaluation.

Preprocessing. The period prefix is stripped from the text (it is bookkeeping noise for the text model), name and description are merged only when they differ, and the date is parsed from the ObjectId.

Features.

Feature	Form	Why
Item text	Word TF-IDF, 1 to 2 word phrases	Keywords such as "subscription", "audit", "training"
Item text	Character TF-IDF, 2 to 5 characters	Robust to typos, SKUs and brand fragments
Vendor	One-hot	Each vendor's usual account
Quarter	One-hot	Lets the model learn the account renames. An unseen quarter gets no columns, a safe fallback
Amount	Sign and power-of-ten bucket	Separates credit notes, small tools and large prepayments without trusting the currency
Months covered	Bucketed count from a `0126–1226` style prefix	Lines spanning many months are usually prepaid, not expensed
Lines in bill	1, 2 to 3, 4 or more	Single-line bills and multi-line bills are coded differently

Memory lookup. Before the model runs, a table of (vendor, cleaned text) to account is built from the training rows. If a line matches and the history is unanimous, the remembered account is used. In cross-validation it answers 47% of lines. It barely changes the accuracy (98.4% on those lines against 98.2% for the model alone), but it gives an auditable reason ("coded like the previous bill") and a well-measured confidence for the gate.

Class imbalance. Class weights are the inverse square root of class frequency. Full balancing over-predicts the tiny classes and costs overall accuracy; no weighting ignores them. Square root is the middle ground. No synthetic oversampling: fake TF-IDF vectors do not correspond to real bills. Singleton accounts stay in the training set so they can be predicted, and their results are reported separately rather than hidden.

Validation. How the model is tested matters as much as how it is built. The rule is that the model is never scored on a bill it has learned from. Two protocols.

1. Primary: five-fold cross-validation, grouped by bill and stratified by account, seed fixed. The memory table, the TF-IDF vocabularies and the class weights are all fitted inside each training fold. Every bill sits in exactly one fold.
2. Secondary: train on bills before 1 September 2025, test on the 1,325 lines after it. This is the question a finance team actually asks, and it exposes new vendors, new wording and a chart-of-accounts change.

The regularisation strength was checked over three values (C of 1, 5 and 20) and the middle value used throughout. The spread was under one point, so the choice adds little optimism. Section 4 lists the other variants that were tried.

4. Results

What it means for the finance team. Out of every 100 lines, the model codes about 89 correctly on its own. It also knows when it is unsure. If only its confident predictions are posted automatically, it can handle 71 of the 100 lines at 98% accuracy, and the other 29 go to a reviewer who sees the right account among three

suggestions 95 times out of 100. The reviewer's job changes from typing an account for every line to confirming a suggestion for one line in three.

Headline metrics. Grouped five-fold cross-validation, out-of-fold predictions on all 4,894 rows. Accuracy is the share of lines coded correctly. Macro F1 averages the score over accounts so that a tiny account counts as much as a huge one; it shows whether the rare accounts are learned or ignored. Weighted F1 is the same but weighted by size. Top-3 accuracy is how often the right account is among the three suggestions.

	Accuracy	Macro F1	Weighted F1	Top-3 accuracy
Grouped 5-fold CV	**0.889**	0.664	0.884	0.953
Time split, Sep 2025 to Feb 2026	0.704	0.584	0.674	0.779

Per-fold accuracy: 0.896, 0.874, 0.897, 0.869, 0.907 (mean 0.889, std 0.014). Resampling the results 2,000 times gives a 95% interval of 0.880 to 0.897, so the true accuracy on similar data is very unlikely to be below 88%.

What each idea bought. Each row adds one thing to the previous one.

Step	Accuracy	Macro F1
Majority class	0.241	
Vendor's most common account (out of fold)	0.689	
Text + vendor + amount, logistic regression	0.855	0.636
+ memory lookup	0.855	0.636
+ quarter	0.883	0.655
+ months covered, lines in bill (final)	0.889	0.664
Same final model, row-level folds (leaky)	0.907	0.674

The quarter feature is worth 2.8 points on its own, almost all of it from the renamed IC Clearing account, whose F1 rises from 0.37 to 0.75 for the new name and from 0.83 to 0.91 for the old one.

What was tried to close the gap to 92%. Each row is a one-line change to the final model (memory lookup left out so only the model changes are compared), scored with the same grouped folds.

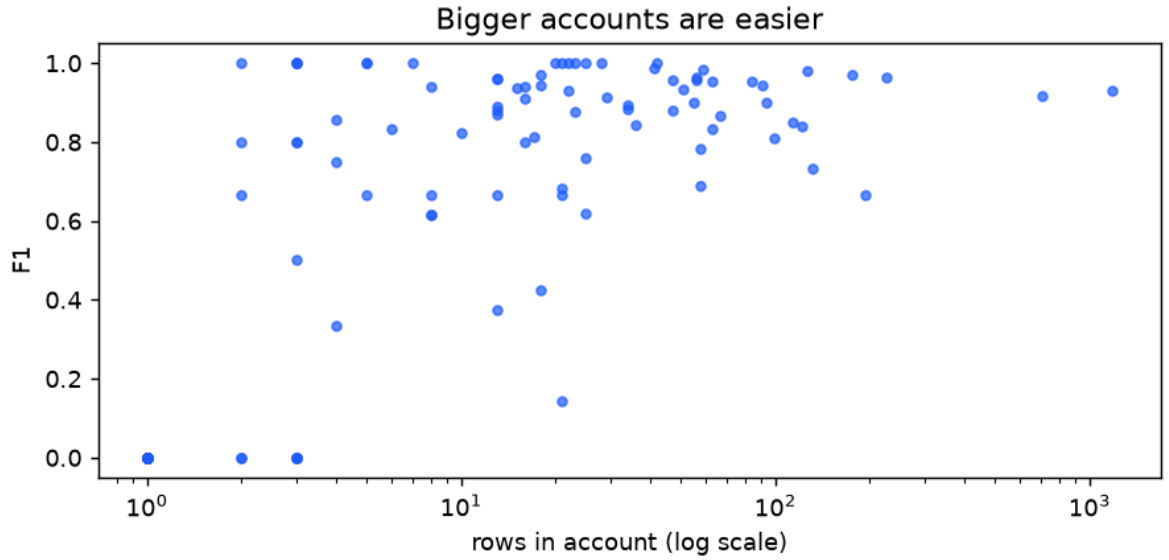
Variant	Accuracy	Macro F1
Final model without memory, C=5	0.887	0.659
C=1 / C=20	0.880 / 0.887	0.653 / 0.655
Linear SVM instead of logistic regression	0.895	0.663
Numeric "months since start" instead of quarter one-hot	0.874	0.646
Vendor-by-word interaction tokens added	0.883	0.655

None of these moves the needle by more than a point, and the SVM gain comes at the cost of the probabilities the confidence gate needs. The numeric date was also checked on the time split, since unlike one-hot quarters it can extrapolate to future periods, and it was slightly worse there too (0.695 against 0.704). The conclusion is that the remaining error is not in the model family but in the information available on the line, which section 5 returns to.

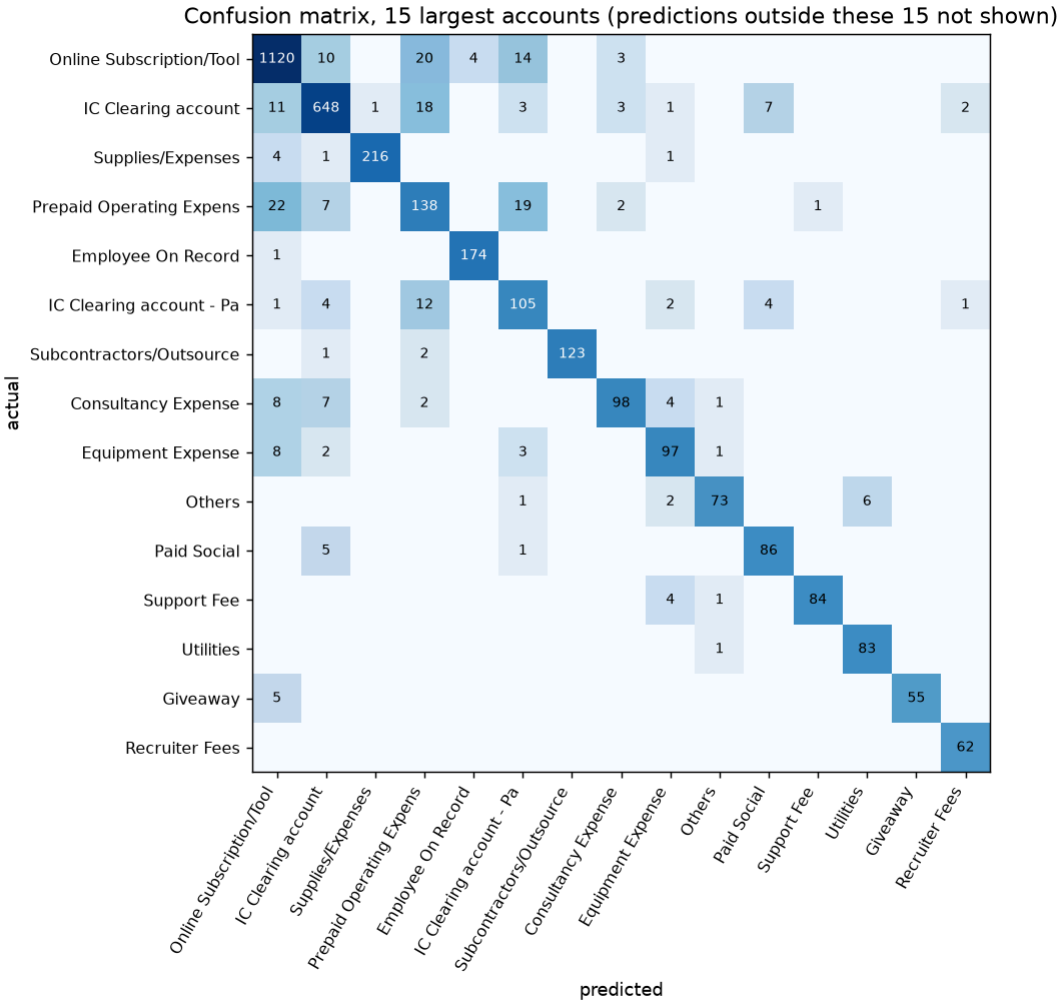
Best and worst accounts. Large, keyword-rich accounts are close to solved: Subcontractors/Outsource (F1 0.98), Employee On Record (0.97), Supplies/Expenses (0.96), Utilities (0.95), Recruiter Fees (0.95), Online Subscription/Tool (0.93). The weak spots are the prepaid family and the "which pocket does this go in" accounts.

Account (rows)	Precision	Recall	F1
134004 Prepaid Subscription (21)	0.29	0.10	0.14
614304 Insurance expenses (13)	1.00	0.23	0.38
134002 Prepaid Insurance (18)	0.47	0.39	0.42
612016 Collateral (25)	0.76	0.52	0.62
134001 Prepaid Operating Expense (193)	0.62	0.72	0.67
617202 Legal Expenses (58)	0.69	0.69	0.69
132098 IC Clearing account - Paid on Behalf (131)	0.68	0.80	0.73

Accounts with fewer than 5 rows have 38% recall in total. They are predicted sometimes, but no claim is made about them.



Error analysis. The confusion matrix of the 15 largest accounts shows three clusters of mistakes, and together they explain most of the remaining error.

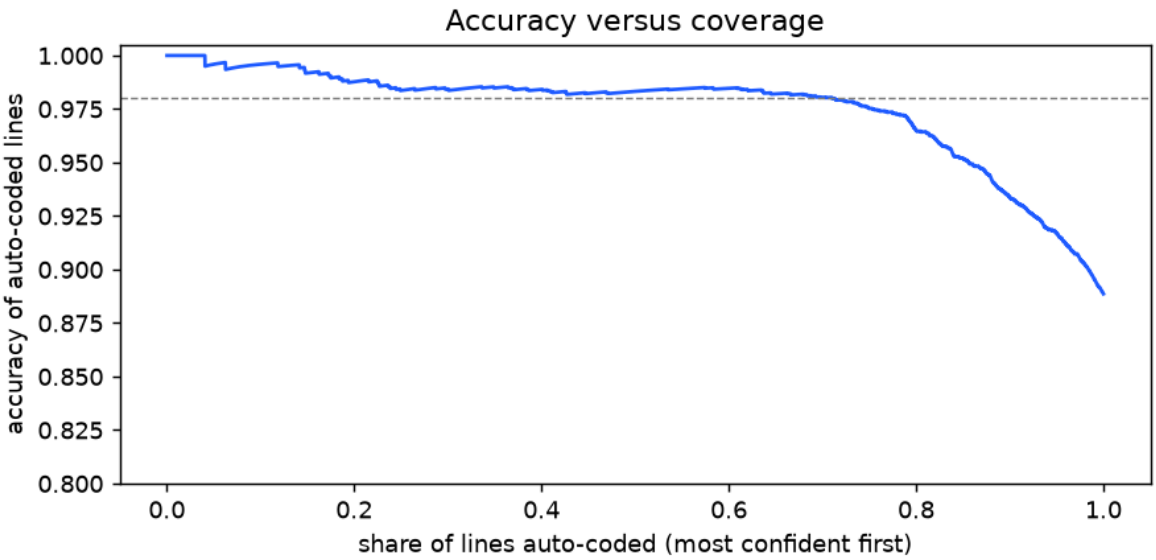


1. **Prepaid versus expense** (Prepaid Operating Expense against Online Subscription/Tool, 42 errors both ways, plus Prepaid Subscription against Prepaid Operating Expense, 17). The same vendor and the same product appear in both, and the difference is an accounting decision about timing. The "months covered" feature helps when the period is written in the item name, but many annual invoices do not say so.
2. **Intercompany versus own expense** (IC Clearing against Prepaid or Subscription, about 60 errors). Whether a SaaS bill is the Singapore entity's own cost or is paid on behalf of a sister company depends on information not in the line, apart from occasional entity tokens like HK or Mudah.
3. **Near-synonym accounts** (Others against Audit Fee, Legal against Secretarial, Collateral against Others). These are judgement calls that the finance team itself may code inconsistently.

Accuracy on the account ID. Scoring on `accountId`, where the old and new names of a renamed account count as one class, gives 0.891 in cross-validation, only 0.2 points above the name-level score. So after adding the quarter feature, renames are no longer a major error source.

The time split, explained. Accuracy falls to 0.704 on the six months after the cutoff. Three things drive the gap. First, 140 test rows (11%) carry an account name that never appears in the training period; 131 of them are "Paid on Behalf", which was created in October 2025. Restricting to names that existed in training gives 0.787, and scoring on `accountId` gives 0.783. Second, 121 test rows come from vendors never seen before, and accuracy on those is 0.231 against 0.752 for known vendors. Third, the memory lookup answers 39% of lines instead of 47%, because fewer lines are recurring over a shorter history. In other words, roughly half of the gap is a chart-of-accounts change, and most of the rest is cold start.

Confidence gate. The model gives each prediction a confidence between 0 and 1. Sorting predictions by that confidence gives the trade-off a deployment would be tuned on: post the confident ones automatically, review the rest.



Required accuracy of auto-coded lines	Share auto-coded	Confidence threshold
99%	18%	0.99
98%	71%	0.95
95%	86%	0.71

At the 98% setting, about seven lines in ten are posted automatically and three go to a reviewer, who sees the correct account among the three suggestions 95% of the time.

5. Discussion

Strengths. The model is simple enough to explain to an auditor and fast enough to retrain every month. The evaluation is the strongest part: grouped folds, out-of-fold memory and vocabulary, a bootstrap interval, a leaky baseline for contrast, and a temporal split with the gap explained rather than hidden. The model is small, trains in seconds, and every prediction has a reason a reviewer can check: a matching previous bill, or the vendor plus the heaviest words. The one real modelling insight, that the account renames make the date a feature, came from the exploration and is worth nearly three points.

Limitations. The 92% internal benchmark was not reached on the fair evaluation. Regularisation, a linear SVM, a numeric date and vendor-by-word interactions were all tried and none closed the gap by more than a point; the leaky row-level evaluation of the same model gives 90.7%, so part of the distance may be a difference in how the two numbers were measured. Rare accounts are essentially unlearnable from one to four examples. The remaining errors are largely accounting judgement (prepaid versus expense, own cost versus intercompany) that the line item alone does not determine. The temporal result shows that the model degrades on new vendors and cannot foresee a chart-of-accounts change. Amounts were not normalised to SGD because the currency is not in the data.

With more time. In rough order of expected value: bring in the bill header (currency, entity, bill date, payment terms) and vendor master data (category, country), which would address both the intercompany and the cold-start problems; add a gradient-boosted model over the same features plus the logistic regression's out-of-fold probabilities, which is the standard way to squeeze another point or two from vendor-by-amount interactions; try a small sentence-embedding kNN for unseen wording, and keep it only if grouped CV improves; and ask finance for a mapping table of renamed accounts so that old and new names are treated as one class from the start.

Deploying this. The right product is not a fully automatic coder but a confidence gate with a review queue. Errors are asymmetric: a mis-coded opex line is a reclassification, while a mis-coded intercompany or prepaid

line affects the balance sheet and month-end close, so the threshold should be stricter for balance-sheet accounts. Corrections from reviewers should flow back into monthly retraining, which is also what keeps the memory lookup growing. New vendors should default to review. Chart-of-accounts changes need a process, not a model: when an account is renamed, the mapping table is updated and the model retrained, otherwise it will keep predicting the old name for a month or two. Finally, each Peakflo client has its own chart of accounts, so this is a per-client model; the reusable asset is the pipeline and the evaluation protocol, not the weights.